

ARX LIFT2s 自有采集、训练与远程推理手册

版本：0.1.0 适用分支：acceptance/official-chain 数据格式：arx_hdf5_v2 HTTP 协议：arx_http_v1
动作语义：state_t_plus_1 固定规格：30 FPS、双臂 14 维 最近更新：2026-09-01
真机状态：新链路离线测试中；部署 body 服务和真机采集前仍需按本文验收。

本文是自有链路的唯一操作源文档。PDF

由本文生成。任何采集、训练、转换、HTTP、推理、高度或启动行为变化，必须在同一提交更新本文和 PDF。

1. 设备、目录与网络

- 设备：ARX LIFT2s, Ubuntu 24.04, ROS 2 Jazzy, ROS_DOMAIN_ID=62。
- 控制 SDK：/home/arx/LIFT, 只读交付目录，不是 Git 仓库。
- 正式仓库：/home/arx/ROS2_LIFT_Play。
- 正式开发分支：acceptance/official-chain；main 只作官方对照。
- 相机：头部 260422272688、左腕 260422274927、右腕 260422274230。
- 模型服务默认地址：http://192.168.31.83:8000，可通过环境变量覆盖。

三路相机、左右臂和 body 必须使用同一 ROS domain。所有正式数据均为双臂 14 维，即使某个任务只活动一只手臂也不改变 schema。

2. 硬件安全与一次性启动

2.1 绝对禁止

- 平台高位时禁止启动、重启、重编译后重启 body。
- 采集和推理脚本不会替操作者启动 body。
- 采集和推理不得自动复位高度。
- 未明确当前高度、目标高度、运动方向和预期运动前，不得发送高度命令。

如果需要部署新的 custom body：先人工缓慢降到安全低位，确认平台已经处于低位，再编译并重启 body。平台高位时保持现有 body 持续运行。

2.2 检查 body 是否已运行

```
source /opt/ros/jazzy/setup.bash
source /home/arx/ROS2_LIFT_Play/custom_sdk/LIFT/body/ROS2/install/setup.bash
ros2 service list | grep lift_height
```

正常应至少包含：

```
/lift_height_lock
/lift_height_set
/lift_height_status
```

服务不存在时，采集和推理直接拒绝启动。不要在平台高位临时重启 body。

3. 显式高度操作

进入环境：

```
cd /home/arx/ROS2_LIFT_Play/act
source /opt/ros/jazzy/setup.bash
```

```
source ../custom_sdk/LIFT/body/ROS2/install/setup.bash
conda activate act
```

查看当前高度、命令高度和锁定状态：

```
python lift_height.py status
```

预演设高，不会运动：

```
python lift_height.py set 15.65
```

允许运动：

```
python lift_height.py set 15.65 --execute
```

工具会显示当前高度、目标高度、UP/DOWN/HOLD 和预期动作，并要求输入完整确认文本。目标范围为 SDK 的 $[0, 20]$ 。设高只修改升降目标，不修改底盘、腰部或头部。

锁定或解锁：

```
python lift_height.py lock
python lift_height.py unlock
```

正式采集和推理前必须处于 locked 状态。

4. 三相机与 VR 检查

VR 正常设备为 `/dev/ttyACM0`，921600 baud；`/ARX_VR_L` 和 `/ARX_VR_R` 正常约 120 Hz。串口持续 0 字节时，先检查转换器到 VR 的 C-to-C 数据线接触。

相机目标为 640x480、90 Hz、USB 3.2。采集器以 30 Hz 选择最新有效帧，不会把相机配置降到 30 Hz。

```
ros2 topic hz /camera/camera_h/color/image_rect_raw/compressed
ros2 topic hz /camera/camera_l/color/image_rect_raw/compressed
ros2 topic hz /camera/camera_r/color/image_rect_raw/compressed
ros2 topic hz /ARX_VR_L
ros2 topic hz /ARX_VR_R
```

5. HDF5 v2 数据采集

先设定任务。任务 slug 用于目录与追踪，自然语言指令用于 VLA：

```
export TASK_NAME=pickplace_right_to_bowl
export TASK_INSTRUCTION="Pick up the object and place it into the bowl."
cd /home/arx/ROS2_LIFT_Play/tools
./01_collect.sh
```

`01_collect.sh` 只启动双臂、相机、VR 和采集器，不启动 body，也不配置 can5。

5.1 按键状态机

- R：开始录制。
- E：结束录制并进入审核。
- S：保存通过校验的 episode。
- D：丢弃当前 episode。
- Q：在非录制状态退出。

录制内容先写到 `act/datasets/.pending/*.hdf5.partial`。只有按 S 后才原子移动为 `episode_N.hdf5` 并占用编号。失败 episode 无法按 S 保存。

5.2 自动拒绝条件

- 三相机缺失、数据年龄超过 50 ms 或相机间时间差超过 20 ms。

- 重复使用同一个相机时间戳。
- episode 丢失采样点超过 1%。
- 实际平均采样频率不在 29.5-30.5 FPS。
- 高度相对开始值变化超过 0.05。
- 任一轮速绝对值超过 0.05。
- state/action 不是有限的 14 维向量。

6. Action 语义与 HDF5 字段

当前 SDK 没有公开内部 joint target，因此当前数据明确使用：

```
observation(t) = 当前双臂关节反馈
action(t)      = state(t+1)
action_semantics = state_t_plus_1
```

采集器会额外取得下一次 30 Hz 状态再完成上一条 transition，不复制最后一帧制造 action。训练 loader 对 arx_hdf5_v2 不再额外移动 action。

主要字段：

```
/observations/qpos
/observations/qvel
/observations/effort
/observations/eef
/observations/images/head
/observations/images/left_wrist
/observations/images/right_wrist
/action
/timestamps/*
/diagnostics/body_information
/diagnostics/wheel_velocity
```

单条校验：

```
cd /home/arx/ROS2_LIFT_Play
conda activate act
python tools/validate_episode.py act/datasets/episode_0.hdf5
```

7. HDF5 转 LeRobot

模型环境固定使用 LeRobot 0.4.3、格式 v3。转换目标目录必须不存在，转换器不会覆盖已有数据。

```
cd /home/arx/ROS2_LIFT_Play
conda activate act
python tools/convert_hdf5_to_learner.py \
  --input act/datasets \
  --output /data/learner/pickplace_right_to_bowl \
  --repo-id local/arx-pickplace-right-to-bowl
```

输出包含三路 RGB 视频、14 维 observation.state、14 维 action 和标准 task/index/timestamp。meta/arx.json 保存原始 HDF5 SHA-256、joint 顺序和 action 语义。

8. 训练

现有 ACT 训练入口会在启动前检查所有 HDF5 文件：

```
cd /home/arx/ROS2_LIFT_Play/tools
./02_train.sh
```

默认要求：

```
expected_action_dim: 14
expected_action_semantics: state_t_plus_1
```

```
expected_fps: 30
```

不同 action 语义的数据混在一起时训练必须失败。训练输出目录包含 `data_contract.yaml`；以后每个 checkpoint 必须与这份契约一起归档。

TAU 训练适配读取 LeRobot 的三路图像、14 维 state、14 维 action 和任务文本。真实 tau-0 模型通过独立 adapter 接入，不改变数据或 HTTP schema。

9. HTTP 模型服务

在模型服务器仓库根目录安装并启动当前 MockPolicy 服务：

```
python -m pip install -r model_server/requirements.txt
python model_server/run_server.py
```

接口：

```
GET /healthz
GET /v1/schema
POST /v1/reset
POST /v1/infer
```

检查：

```
curl http://192.168.31.83:8000/healthz
curl http://192.168.31.83:8000/v1/schema
```

MockPolicy 只返回保持当前关节位置的 action chunk，用于链路测试。TauPolicyAdapter 在接入真实模型前会明确抛出 `NotImplementedError`。

10. ARX 远程推理

默认是 dry-run，不发布机械臂 action：

```
export TASK_INSTRUCTION="Pick up the object and place it into the bowl."
export MODEL_SERVER_URL=http://192.168.31.83:8000
cd /home/arx/ROS2_LIFT_Play/tools
./03_inference.sh
```

Client 会检查 body 高度、HTTP schema、14 维 action、action 语义、NaN/Inf、500 ms 响应时限和机身运动。掉线或异常后不会自动重新 armed。

真机 `--execute` 还必须同时提供：

- 经过实机审核的 14 维 joint limits YAML。
- `--confirm-execute I_UNDERSTAND`。
- 首动作与逐步动作变化通过安全门。

仓库中的 `joint_limits.example.yaml` 故意为空，不能用于真机执行。

11. 故障与恢复

相机或 ROS 数据中断

当前 tick 被丢弃；超过 1% 时整条 episode 不可保存。检查 USB 3.2、相机序列号和 ROS domain。

机身在 episode 中移动

按 D 丢弃。不要通过修改阈值掩盖实际移动。

留下 partial 文件

partial 文件不计入 episode 编号。确认是崩溃残留且不需恢复后，再人工移出 .pending；不要改名冒充正式 episode。

HTTP 超时或掉线

推理 client disarm 并停止新 action。修复网络后重新启动完整 session，不自动续接旧 session。

需要重启 body

先缓慢降到安全低位，再重启。平台高位时禁止操作。

12. 迁移到真实 joint target

SDK 工程师提供只读 joint-target 话题后：

1. 新增 RosJointCommandActionProvider。
2. action 语义改为 joint_position_command，offset 改为 0。
3. 建立新的数据集、统计量和 checkpoint。
4. 禁止与 state_t_plus_1 数据合并。
5. 同步更新本文、PDF、转换器契约和 HTTP schema 版本。

13. 验收记录

- 2026-08-31：官方链路完成三条 random smoke；仅证明旧 HDF5、ACT 和 GPU 可运行，不得部署。
- 2026-09-01：自有 HDF5 v2、HTTP、转换与文档开始实施；离线测试和新 body 编译结果以仓库测试报告为准。
- 待执行：安全低位部署新 body 服务、三条 pilot、主动丢弃测试、机身移动拒绝测试、HTTP dry-run 和真实 tau-0 adapter 验收。